# Operators in PQL

Was this page helpful?Â

This topic gives you an overview of the arithmetic and boolean operators that you can use in PQL expressions, and the precedence of all operators.

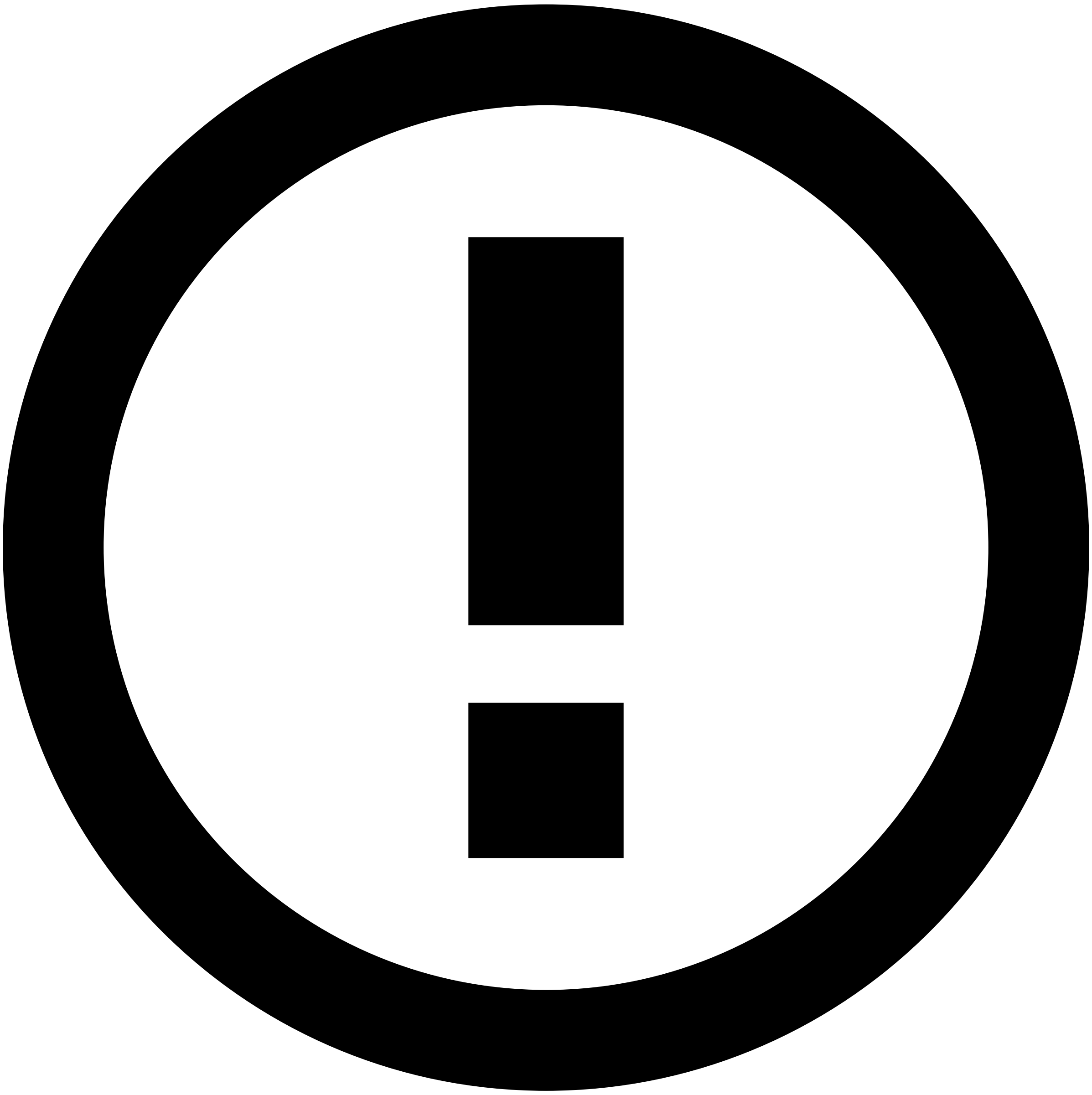
## Arithmetic operatorsâ

PQL uses the following arithmetic operators:

| Operator | Meaning |
| --- | --- |
| `+` | addition |

| Operator | Meaning |
|---|---|
| `-` | subtraction |
| `*` | multiplication |
| `/` | division |
| `%` | integer remainder |
| `^` | exponentiation |

Example: `Balance - Amount`



Type promotion when using arithmetic operators

The arithmetic operators obey the following type promotion rules:

1. If at least one of the arguments is a double, the result is a double.
2. If at least one of the arguments is a float, the result is a float, unless the result is a double because of the first rule.

3. If at least one of the arguments is a long, the result is a long, unless the result is a double or float because of the first two rules.

## Boolean operatorsâ

Boolean operators are operators that return a boolean value. They include:

- `<` , `<=` , `==` , `!=` , `>=` , `>`

info

Only numeric values can be compared with the <, <=, >=, and > operators.

- `and` , `or` , `!` (not)

Example:

```
POSEntryMode == 'Swiped' or POSEntryMode == 'Keyed' or POSEntryMode == 'Chip'
```

## Operator precedenceâ

The following table lists PQL operators according to their precedence order. Operators at the top of the table have higher precedence.

| Operator type | Operator(s) |
|---|---|
| grouping | `(expr)` |
| unary | `+expr` , `-expr` , `!` |
| arithmetic - exponential | `^` |
| cast | `exp:type` |
| arithmetic âmultiplicative | `*` , `/` , `%` |
| arithmetic âadditive | `+` , `-` |
| coalesce | `??`<br><br>Note that this operator has a different precedence in RDL. |
| comparison | `<` , `<=` , `==` , `>=` , `>` , `!=` |
| logical âAND | `and` |
| logical âOR | `or` |
| assignment | `=` |

Cast operator precedence

Notice that the *cast* operator precedence lies between arithmetic operators. If an expression needs to be cast before an exponentiation, it needs to be enclosed in parentheses (*grouping* operator).

The following rules apply:

- operators with higher precedence than other operators are evaluated before them;
- operators on the same line have the same precedence:
    - all binary operators except for the assignment operator are evaluated from left to right;
    - assignment operator is evaluated right to left.

## Examplesâ

Arithmetic and boolean operators of PQL follow the common rules of mathematics regarding functionality and precedence.

Arithmetic and boolean operators have higher precedence than the `??` (coalesce) operator, meaning that the `??` operator gets evaluated after arithmetic and boolean operators.

Example: `a + b ?? c + d` is equivalent to `(a+b) ?? (c+d)`.